



Architecture monolithique

Le Framework Spring Boot

Chapitre 7 : Spring Data et JPA

Sabeur ELKOSANTINI
Maître de conférences en informatique
Sabeur.Elkosantini@fsegn.ucar.tn
<https://sabeur.elkosantini.me>

Plan

- Chapitre 1 : Introduction aux architectures monolithiques
- Chapitre 2 : Framework SpringBoot : concepts de base
- Chapitre 3 : Framework SpringBoot : Spring Data JPA
- Chapitre 4 : Framework SpringBoot : Les app. monolithiques
- Chapitre 5 : Framework SpringBoot : La configuration
- Chapitre 6 : Framework SpringBoot : Session et sécurité
- Chapitre 7: Framework SpringBoot : Les formulaires

Frameworks MVC

👉 Le framework Spring boot

- Modèle et persistance des données

Modèle DAO:

- Package pour les modèles (les Beans ou les entités JPA)
- Couche DAO (ou repository dans spring)

Ajouter les dépendances MySQL et Spring-Data-JPA

Persistance en Java

👉 ORM : hibernate

- Persistance automatisée et transparente d'objets métiers vers une bases de données relationnelles,
- Description à l'aide de méta-données de la transformation réversible entre un modèle relationnel et un modèle de classes
- Capacité à manipuler des données stockées dans une base de données relationnelles à l'aide d'un langage de programmation orientée-objet
- Techniques de programmation permettant de lier les bases de données relationnelles aux concepts de la programmation OO pour créer une "base de données orientées-objet virtuelle"

Persistence en Java

👉 ORM: hibernate

- Une couche d'abstraction a la base de données
- Permet à l'utilisateur d'utiliser les tables d'une base de données comme des objets
- consiste a associer :
 - ✓ une classe à chaque table
 - ✓ un attribut de classe a chaque colonne de la table
- A comme objectif de ne plus écrire de requête SQL
- Supprime une très grande partie du code nécessaire avec JDBC

Persistence en Java

👉 ORM: hibernate

ORM : avantages

- Suppression d'une très grande partie du code nécessaire avec JDBC
 - Gain de temps
 - Simplification du code source
- Code portable indépendant vis-à-vis de SGBD

Persistence en Java

👉 Hibernate

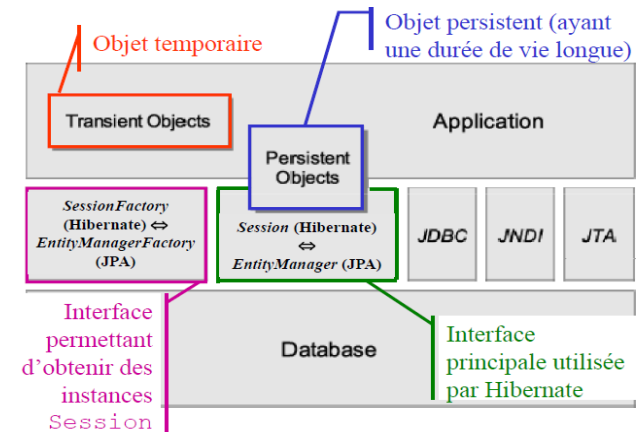
- Est un framework ORM pour Java
- Hibernate s'occupe du transfert des classes Java dans les tables de la base de données (et des types de données Java dans les types de données SQL)

Quel avantage est apporté par Hibernate ?

- Il réduit de manière significative le temps de développement qui aurait été autrement perdu dans une manipulation manuelle des données via SQL et JDBC.

Persistence en Java

👉 Hibernate : architecture

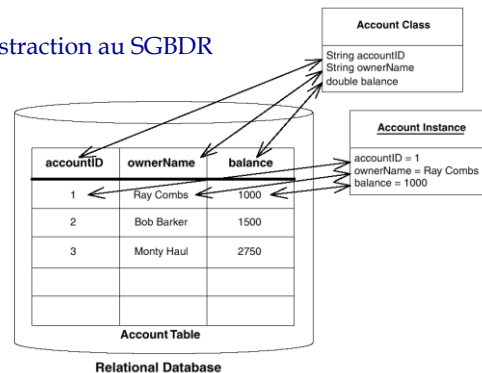


Hibernate

🔑 Hibernate : architecture

- ORM : Object Relational Mapping:
 - ✓ Relier les mondes objet et relationnel
 - ✓ Offrir une couche d'abstraction au SGBDR

Offre une couche d'abstraction
au SGBDR



S. Elkosantini

9

Frameworks MVC

🔑 JPA : Configuration

- Modèle et persistance des données

Configuration du `application.properties` pour la connexion au SGBD

```
spring.datasource.url=jdbc:mysql://localhost:3306/nom_de_ta_b
ase?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

S. Elkosantini

10

Hibernate

🔑 Configuration

- Les valeurs de `spring.jpa.hibernate.ddl-auto` :

- `create` : crée les tables, les données précédemment présentes dans les tables seront perdues.
- `update` : met à jour les tables avec les valeurs données. si les tables ne sont pas présentes dans la base de données, elles seront créées.
- `validate` : si les tables ne sont pas présentes dans la base de données, elles ne seront pas créées et une exception sera levée.
- `create-drop` : créer les tables en détruisant les données précédemment présentes. Les tables de la base de données seront aussi supprimées lorsque la SessionFactory est fermée (à utiliser pour tester).

Hibernate

🔑 Configuration des beans

- Création des JavaBeans: Entités et composants

```
package tn.entites;  
  
import java.io.Serializable;  
import javax.persistence.*;
```

```
@Entity  
@Table(name = "etudiant")  
public class Student {
```

```
    @Id  
    private int sid;  
    public int getSid() {  
        return sid;  
    }  
    public void setSid(int sid) {  
        this.sid = sid;  
    }  
}
```

Les annotations

Hibernate

☞ Configuration des beans

■ Création des JavaBeans: Entités et composants

```
@Column
String name;
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}

@Column
int score;
public int getScore() {
    return score;
}
public void setScore(int score) {
    this.score = score;
}
}
```

S. Elkosantini

13

13

Hibernate

☞ Configuration des beans

■ Création des JavaBeans: : Entités et composants

○ Quelques annotations (pour le mapping) :

✓ **@Entity**: indique que les instances de la classe sont persistantes

(stockées dans la base);

➔ L'implémentation JPA, qui est Hibernate, dans ce cas, pourra effectuer des opérations CRUD sur les entités du domaine

✓ **@Table(name = "student")**: indique que le nom de la table

S. Elkosantini

14

14

Hibernate

☞ Configuration des beans

■ Création des JavaBeans : Entités et composants

- Quelques annotations (pour le mapping) :
 - ✓ **@Id**: indique que cette propriété est la clé primaire. Deux stratégies possibles (**@GeneratedValue**)
 - **Strategy = GenerationType.AUTO**. Hibernate produit lui-même la valeur des identifiants.
 - **Strategy = GenerationType.IDENTITY**. Hibernate s'appuie alors sur le mécanisme propre au SGBD pour la production de l'identifiant

Attention à la méthode setId

Hibernate

☞ Configuration des beans

■ Création des JavaBeans: Entités et composants

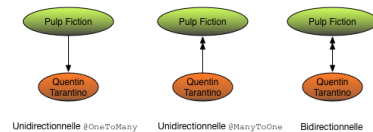
- Quelques annotations (pour le mapping) :
 - ✓ **@Column**: indique que la propriété est associée à une colonne de la table. Elle peut prendre les paramètres suivants:
 - **name** : Pour indiquer le nom du champ de la table si l'attribut est différent
 - **length** indique la taille maximale de la valeur de la propriété;
 - **nullable** (avec les valeurs false ou true) indique si la colonne accepte ou non des valeurs à NULL;
 - **unique** indique que la valeur de la colonne est unique.

Hibernate

🔑 Configuration des beans

■ Création des JavaBeans:: Les associations

✓ Associations un-à-plusieurs:



- Dans un film, on place un lien vers le réalisateur (unidirectionnel, à gauche);
- Dans un artiste, on place des liens vers les films qu'il a réalisés (unidirectionnel, centre);
- On représente les liens des deux côtés (bidirectionnel, droite).

Le problème ne se pose pas dans les bases de données

Hibernate

🔑 Configuration des beans

■ Création des JavaBeans:: Les associations

✓ Associations un-à-plusieurs

✓ Les annotations possibles sont:

- @ManyToOne
- @OneToMany
- @JoinColumn(name="idClasse") : indique la clé étrangère

Hibernate

☞ Configuration des beans

■ Création des JavaBeans:: Les associations

- ✓ Associations un-à-plusieurs
- ✓ Les annotations possibles sont:

- @ManyToOne

De côté entité Student

```
@ManyToOne
@JoinColumn(name="idClasse")
private Classe classe;
```

Il faut créer l'entité Classe

Hibernate

☞ Configuration des beans

■ Création des JavaBeans:: Les associations

- ✓ Associations un-à-plusieurs
- ✓ Les annotations possibles sont:

- @OneToMany

De côté entité Classe

```
@OneToMany
@JoinColumn(name="idClasse")
private List<Student> listeS = new ArrayList<Student>();
public void addStudent(Student st) {listeS.add(st) ;}
public List<Student> getListe() {return listeS;}
```

Quels problèmes avec cette solution ?

Hibernate

☞ Configuration des beans

■ Création des JavaBeans:: Les associations

- ✓ Associations bidirectionnelles **Ce n'est pas la bonne solution**
- ✓ Les annotations possibles sont: **De côté entité Student**

```
De côté entité Classe
@OneToMany
@JoinColumn(name="idClasse")
private Classe classe;

De côté entité Student
@ManyToOne
@JoinColumn(name="idClasse")
private List<Student> listeS = new ArrayList<Student>();
public void addStudent(Student st) {listeS.add(st) ;}
public List<Student> getListe() {return listeS;}
```

Attention à l'ajout de l'étudiant (à voir plus tard dans la couche DAO)

Hibernate

☞ Configuration des beans

■ Création des JavaBeans:: Les associations

- ✓ Associations bidirectionnelles
- ✓ Les annotations possibles sont:

```
De côté entité Classe
@OneToMany
@JoinColumn(name="idClasse")
private List<Student> listeS = new ArrayList<Student>();

De côté entité Classe
@OneToMany(mappedBy="classe")
private List<Student> listeS = new ArrayList<Student>();
```

Le nom de l'attribut de l'autre classe

Il faut penser à améliorer la méthode d'insertion d'un nouveau étudiant dans la couche DAO

Hibernate

🔑 Configuration des beans

■ Création des JavaBeans:: Les associations

- ✓ Associations plusieurs-à-plusieurs : relation sans attributs



De côté entité Student (qui détient la relation)

```
@ManyToMany()
@JoinTable(name="Student_Classe", joinColumns = @JoinColumn(name = "idStudent", inverseJoinColumns = @JoinColumn(name = "idClasse"))
List<Classe> listeC = new ArrayList<Classe>();
```

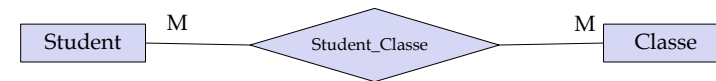
Nom de la table (association) Clé étrangère de la table Student
Clé étrangère de la table Classe

Hibernate

🔑 Configuration des beans

■ Création des JavaBeans:: Les associations

- ✓ Associations plusieurs-à-plusieurs : relation sans attributs



De côté entité Classe

Le nom de l'attribut représentant l'association dans la classe Student

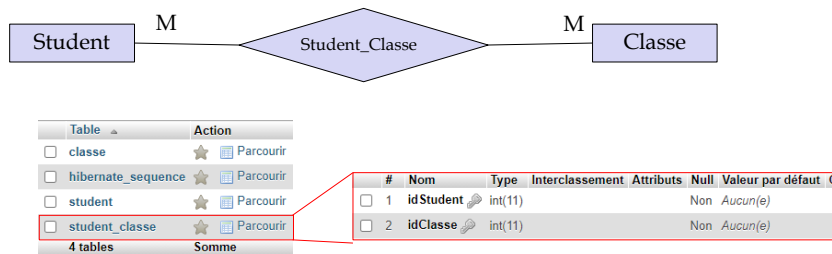
```
@ManyToMany(mappedBy="listeC")
List<Student> listeS = new ArrayList<Student>();
```

Hibernate

Configuration des beans

■ Création des JavaBeans:: Les associations

- ✓ Associations plusieurs-à-plusieurs : relation sans attributs

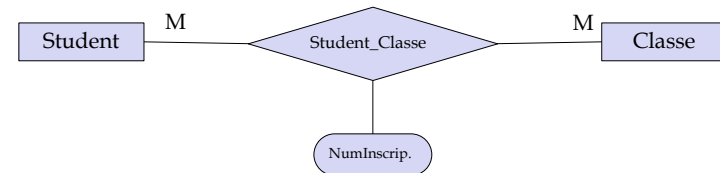


Hibernate

Configuration des beans

■ Création des JavaBeans:: Les associations

- ✓ Associations plusieurs-à-plusieurs : relation avec attributs



Hibernate

🔑 Configuration des beans

■ Création des JavaBeans:: Les associations

✓ Associations plusieurs-à-plusieurs : relation avec attributs

- Etape 1: créer la table de jointure

Comment implémenter les clés composées ?

```
@Entity
@Table(name = "Inscription")
public class Inscription {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int idI;

    @ManyToOne
    @JoinColumn(name = "idClasse")
    private Classe classe;

    @ManyToOne
    @JoinColumn(name = "idStudent")
    private Student student;
```

← Générer une clé simple

Hibernate

🔑 Configuration des beans

■ Création des JavaBeans:: Les associations

✓ Associations plusieurs-à-plusieurs : relation avec attributs

- Etape 1: créer la table de jointure

Comment implémenter les clés composées ?

Coté entité Student

```
@OneToMany (mappedBy="student")
private List<Inscription> listeI = new
ArrayList<Inscription>();
```

La relation devient OneToMany avec Mapping avec l'attribut student de la relation

Hibernate

👉 Configuration des beans

■ Création des JavaBeans:: Les associations

✓ Associations plusieurs-à-plusieurs : relation avec attributs

- Etape 1: créer la table de jointure

Comment implémenter les clés composées ?

Coté entité classe

```
@OneToMany(mappedBy="classe")  
private List<Inscription> listeI= new ArrayList<Inscription>();
```

La relation devient OneToMany avec Mapping avec l'attribut classe de la relation

Hibernate

👉 Configuration des beans

■ Création des JavaBeans:: Les associations

✓ Associations plusieurs-à-plusieurs : relation avec attributs

- Etape 1: créer la table de jointure

Comment implémenter les clés composées ?

Utiliser les composants en utilisant @Embeddable (et non une entité)

```
@Embeddable  
public class InscriptionId implements Serializable {  
  
    @ManyToOne  
    @JoinColumn(name = "idClasse")  
    private Classe classe;  
  
    @ManyToOne  
    @JoinColumn(name = "idStudent")  
    private Student student;
```

La clé composée

Hibernate

🔑 Configuration des beans

■ Création des JavaBeans:: Les associations

✓ Associations plusieurs-à-plusieurs : relation avec attributs

- Etape 1: créer la table de jointure

Comment implémenter les clés composées ?

La table de jointure

```
@Entity
@Table(name = "Inscription")
public class Inscription {

    @Id
    private InscriptionId Ins = new InscriptionId() ;
```

Hibernate

🔑 Configuration des beans

■ Création des JavaBeans:: Les associations

✓ Associations plusieurs-à-plusieurs : relation avec attributs

- Etape 2: Modifier les autres entités

Comment implémenter les clés composées ?

Attention aux entités Student et Classe

L'id de la relation

```
@OneToMany (mappedBy="Ins.student")
private List<Inscription> ListeI = new ArrayList<Inscription>();

@OneToMany(mappedBy="Ins.classe")
private List<Inscription> ListeI= new ArrayList<Inscription>();
```


Hibernate

👉 Configuration des beans

- Gestion des relations avec les annotations de cascade
 - ✓ Les cascades permettent de propager les opérations (save, delete, etc.) d'une entité parent vers ses entités liées.
 - ❖ Exemple : Sauvegarder une Classe et ses Student en une seule opération.
 - ✓ Types de CascadeType
 - ❖ **PERSIST** : Persiste les entités liées.
 - ❖ **MERGE** : Met à jour les entités liées.
 - ❖ **REMOVE** : Supprime les entités liées.
 - ❖ **ALL** : Applique toutes les opérations en cascade.
 - ❖ **DETACH**, **REFRESH** : Moins courants, mais utiles dans des cas spécifiques.

Hibernate

👉 Configuration des beans

- Gestion des relations avec les annotations de cascade

Exemple

```
@Entity
@Table(name = "classe")
public class Classe {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int idClasse;
    private String nom;

    @OneToMany(mappedBy = "classe", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Student> students = new ArrayList<>();
}
```

cascade = CascadeType.ALL : Si une Classe est persistée, tous ses Student le sont aussi. Si une Classe est supprimée, ses Student le sont également.

orphanRemoval = true : Si un Student est retiré de la liste students, il est supprimé de la base de données.



Est-ce correct, orphanRemoval = true ?

Hibernate

👉 Configuration des beans

- Gestion du chargement des relations avec *FetchType*
 - ✓ FetchType contrôle quand et comment ces données liées sont chargées depuis la base de données :
 - ❖ **Eager Loading** (Chargement immédiat) : Les données liées sont récupérées en même temps que l'entité principale.
 - ❖ **Lazy Loading** (Chargement paresseux) : Les données liées ne sont chargées que lorsqu'on y accède explicitement.



Objectif : Optimiser les performances en évitant de charger inutilement des données.

Hibernate

👉 Configuration des beans

- Gestion du chargement des relations avec *FetchType*
 - ✓ FetchType.EAGER :
 - ❖ Charge immédiatement les entités liées.
 - ❖ Par défaut pour @ManyToOne et @OneToOne.



Peut entraîner des performances médiocres si les relations sont volumineuses.

```
@Entity
@Table(name = "student")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int sid;
    private String name;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "idClasse")
    private Classe classe;
```

Quand on charge un **Student**, l'objet **Classe** associé est récupéré immédiatement dans la même requête.

Hibernate

👉 Configuration des beans

■ Gestion du chargement des relations avec *FetchType*

✓ FetchType.LAZY :

- ❖ Charge les entités liées uniquement à la demande (quand on appelle un getter, par exemple).
- ❖ Par défaut pour @OneToMany et @ManyToMany.

```
@Entity
@Table(name = "classe")
public class Classe {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int idClasse;
    private String nom;

    @OneToMany(mappedBy = "classe", cascade = CascadeType.ALL, fetch =
    FetchType.LAZY)
    private List<Student> students = new ArrayList<>();
```

Quand on charge une Classe, la liste students n'est pas récupérée immédiatement. Elle est chargée uniquement si on appelle classe.getStudents().

Hibernate

👉 Configuration des beans

■ Bonnes pratiques avec les cascades:



Utiliser `CascadeType.ALL` avec prudence pour éviter des suppressions ou mises à jour involontaires.



Préférer `FetchType.LAZY` pour les relations (ex. : `@OneToMany(fetch = FetchType.LAZY)`) et charger les données liées explicitement si nécessaire.

Hibernate

👉 La validation des entités

- Ajout de la dépendance de validation:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Validation lors de l'ajout de l'objet

```
@NotEmpty(message = "Le nom est obligatoire")
@Size(min = 2, max = 50, message = "Le nom doit contenir entre 2 et 50 caractères")
@Column(nullable = false, length = 50)
private String nom;
```

Lors de la création du schéma de la base

Hibernate

👉 La validation des entités

- Autres exemples :

Le login doit avoir des caractères minuscules et majuscules et des chiffres de 0 à 9

```
@Pattern(regexp = "[A-Za-z0-9]+")
private String login;
```

Valeur max et min

```
@Min(18)
@Max(45)
private int age;
```

```
@Column(nullable = false, length = 50)
private String prenom;
```

Doit être un email

```
@Email
@Column(nullable = false, length = 70, unique = true)
private String email;
```

Hibernate

☞ La validation des entités

- Validation de groupes et des relations :
Propagation de la validation

```
@Valid
@ManyToOne
@JoinColumn(name="idGroupe")
private Groupe gr;

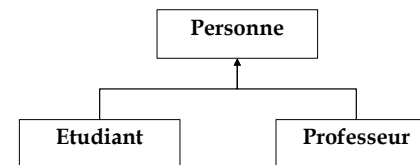
@NotEmpty(message = "un groupe doit avoir au moins un étudiant")
@Size(min = 1, max = 30, message = "Un groupe peut avoir entre (min) et (max) étudiants")
@OneToMany(mappedBy = "gr")
private List<@Valid Groupe> listeG = new ArrayList<Groupe>();
```

Propagation de la validation

Hibernate

☞ Implémentation de l'héritage

- JPA propose 3 stratégies d'implémentation:
 - ✓ Héritage par une seule table
 - ✓ Héritage avec jointure de table
 - ✓ Héritage avec une table par classe



Hibernate

🔑 Implémentation de l'héritage

- JPA propose 3 stratégies d'implémentation:

- ✓ Héritage par une seule table

Stratégie de l'héritage et la colonne qui définit l'héritage

```
@Data
@Entity
@Inheritance(strategy=InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="Personne_type")
public abstract class Personne {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;
    private String nom;
}
```



Résultat : une seule table

note	specialite	id	personne_type	nom
------	------------	----	---------------	-----

```
@Data
@Entity
@DiscriminatorValue("ETU")
public class Enseignant extends Personne {
    private int Specialite;
}
```

Valeur de la colonne Personne_Type

```
@Data
@Entity
@DiscriminatorValue("ENS")
public class Enseignant extends Personne {
    private int Specialite;
}
```

Hibernate

🔑 Implémentation de l'héritage

- JPA propose 3 stratégies d'implémentation:

- ✓ Héritage avec jointure de table

```
@Data
@Entity
@Inheritance(strategy=InheritanceType.JOINED)
@DiscriminatorColumn(name="Personne_type")
public abstract class Personne {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;
    private String nom;
}
```

```
@Data
@Entity
@DiscriminatorValue("ETU")
public class Enseignant extends Personne {
    private int Specialite;
}
```

```
@Data
@Entity
@DiscriminatorValue("ENS")
public class Enseignant extends Personne {
    private int Specialite;
}
```

enseignant	Parcourir	Structure	Rechercher	Insérer	Vider	Supprimer
enseignant						
etudiant						
personne						

id	personne_type	nom
note	id	specialite

Clé étrangère

Hibernate

🔑 Implémentation de l'héritage

- JPA propose 3 stratégies d'implémentation:

- ✓ Héritage avec une table par classe

```
@Data
@Entity
@Inheritance(strategy=InheritanceType.TABLE_PER_CLASS)
public abstract class Personne {
    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private Long id;
    private String nom;
}
```

```
@Data
@Entity
@Table(name="Enseignant")
public class Enseignant extends Personne {
    private int Specialite;
}
```

```
@Data
@Entity
@Table(name="Etudiant")
public class Etudiant extends Personne {
    private int note;
}
```



Pas de @DiscriminatorColumn, pas GenerationType.IDENTITY, ni de @DiscriminatorValue ("ENS")

Hibernate

🔑 Implémentation de l'héritage

- JPA propose 3 stratégies d'implémentation : Comparaison

	SINGLE_TABLE	JOINED	TABLE_PER_CLASS
Avantages	- Requêtes simples - Meilleures performances en lecture - Pas de JOIN	- Normalisation élevée - Pas de colonnes NULL - Respect des principes relationnels	- Indépendant par classe - Pas de couplage entre entités - Bon pour hiérarchies disjointes
Inconvénients	- Colonnes NULL pour les champs spécifiques - Moins normalisé - Table peut devenir large	- Requêtes avec JOIN - Performances en lecture plus lentes - Plus complexe à gérer	- Requêtes UNION implicites - Mauvaises performances - Pas de partage de PK possible - Polymorphisme limité



Recommandée: Normalisation, Intégrité des données

Frameworks MVC

👉 Le framework Spring boot

- Modèle et persistance des données
 - Couche DAO (ou repository dans spring)
- Il faut créer une interface qui étend:
 - ✓ soit *CrudRepository* : fournit les méthodes principales pour le CRUD
 - ✓ soit *PagingAndSortingRepository* : hérite de *CrudRepository* et fournit en plus des méthodes de pagination et de tri sur les enregistrements
 - ✓ soit *JpaRepository* : hérite de *PagingAndSortingRepository* en plus de certaines autres méthodes JPA.

Frameworks MVC

👉 Le framework Spring boot

- Modèle et persistance des données
 - Couche DAO (ou repository dans spring)
 - Dans le contrôleur:
 1. Ajouter une méthode dans le contrôleur pour l'affiche du formulaire
 2. Ajouter une méthode pour l'action du formulaire

Frameworks MVC

👉 Le framework Spring boot

■ Modèle et persistance des données

• Exemples de méthode du repository:

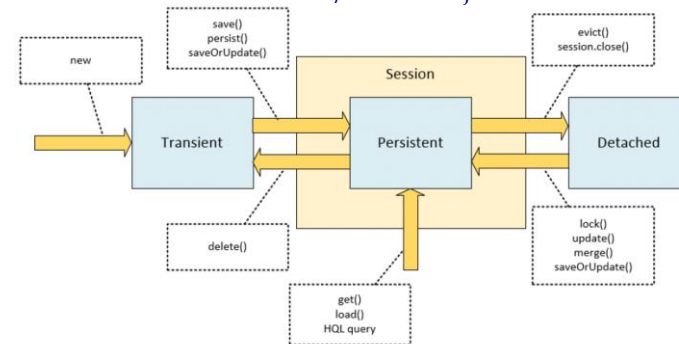
- save(Object): enregistrer l'objet
- findAll(): chercher tous les objets
- findById(): recherche selon la valeur de la clé primaire
- findAllById(): recherche selon un tableau de clé primaire
- deleteById(): Supprimer selon la valeur de la clé primaire
- deleteAll(): supprimer tout
- flush(): modifier
- count(), exists(), existsById()...

Hibernate

👉 Configuration des beans

■ La couche DAO

✓ Insertion : autres méthodes/état des objets



Hibernate

☞ Configuration des beans

■ La couche DAO

✓ Les états des objets

➤ Éphémère (transient)

- un objet est éphémère s'il a juste été instancié.
- Il n'a aucune représentation persistante dans la base de données et aucune valeur d'identifiant n'a été assignée.
- Les instances éphémères seront détruites par le ramasse-miettes si l'application n'en conserve aucune référence.

Hibernate

☞ Configuration des beans

■ La couche DAO

✓ Les états des objets

➤ Persistant

- Une instance persistante a une représentation dans la base de données et une valeur d'identifiant
- Hibernate détectera n'importe quels changements effectués sur un objet dans l'état persistant et synchronisera l'état avec la base de données lors de la fin l'unité de travail,

Hibernate

☞ Configuration des beans

■ La couche DAO

✓ Les états des objets

➤ Détaché

- une instance détachée est un objet qui a été persistant, mais dont sa Session a été fermée.
- La référence à l'objet est encore valide et l'instance détachée pourrait même être modifiée dans cet état.
- Une instance détachée peut être réattachée à une nouvelle Session plus tard.

Comment passer entre les états

Hibernate

☞ Configuration des beans

■ La couche DAO : save Vs persist

- `save` est une méthode **Hibernate** tant dis que `persist` est une méthode **JPA**
- `save` retourne la valeur de la clé primaire attribuée au tuple
- `persist` n'a pas de valeur de retour
- `save` attribue l'identifiant au tuple immédiatement (à l'exécution d'`insert`) même s'il n'est pas dans une transaction car il s'agit bien de sa valeur de retour
- `persist` attribue l'identifiant au tuple à la fin de la transaction ou lorsqu'on fait un `flush()`
- `persist` peut appliquer la persistance en cascade
- `persist` et `save` permettent aussi de faire la modification si la clé de l'objet à ajouter existe dans la base de données

Hibernate

☞ Configuration des beans

■ La couche DAO

✓ Les états des objets

➤ Persistance: `session.merge(objet)`

- Fusionne une instance détachée avec une instance persistante (existante ou chargée depuis la base)
- Effectue un SELECT avant pour déterminer s'il faut faire INSERT ou UPDATE

Hibernate

☞ Configuration des beans

■ La couche DAO

✓ Les états des objets

➤ Persistance: `session.update(objet)`

- Force la mise à jour (UPDATE) de l'objet dans la base
- Lève une exception si une instance de même identificateur existe dans la Session

Hibernate

☞ Configuration des beans

■ La couche DAO

✓ Les états des objets

➤ Détachement

- Plus de surveillance de l'instance par la Session :
- Plus aucune modification rendue persistante de manière transparente
- Trois moyens de détacher une instance :
 - ❖ En fermant la session : `session.close()`
 - ❖ En vidant la session : `session.clear()`
 - ❖ En détachant une instance particulière: `session.evict(objet)`

Frameworks MVC

☞ Le framework Spring boot

■ Modèle et persistance des données

• Exemple 1

```
@PostMapping(value = "/addProduit")
public ModelAndView AjouterProduit(@RequestParam(value="id") String id,
    @RequestParam(value="nom") String nom, @RequestParam(value="qte") int qte,
    @RequestParam(value="idc") String idc) {
    Produit p = new Produit(id, nom, qte, idc);
    produitrepository.save(p);
    ModelAndView mv= new ModelAndView("confirm");
    mv.addObject("id", id);
    mv.addObject("nom", nom);
    return mv;
}
```

Lecture des paramètres de l'URL

Enregistrement

Frameworks MVC

👉 Le framework Spring boot

■ Modèle et persistance des données

• Exemple 2

```
@GetMapping(value="/afficheListe")
public ModelAndView afficheListe() {

    ModelAndView mv = new ModelAndView("AfficheListe");
    //List<Produit> liste = (ArrayList<Produit>)produitrepository.findAll();
    List<Produit> liste = produitrepository.findAll(Sort.by("id").descending());

    mv.addObject("produits",liste);
    return mv;
}
```

Résultat trié

Frameworks MVC

👉 Le framework Spring boot

■ Modèle et persistance des données

• Requêtes personnalisées dans le repository

1. Ajouter de nouvelles méthodes

```
public interface ProduitRepository extends JpaRepository
<Produit, Integer> {
    List<Produit> findByIdAndNom(String id, String nom);
    List<Produit> findByQteLessThan(int qte);
}
```

Recherche par id ET nom

Recherche par quantité < qte

Dans le contrôleur :

```
List<Produit> liste = produitrepository.findByQteLessThan(11);
```

Pour les requêtes JPA: Consulter <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/#jpa.query-methods.query-creation>